

Module: Command Line arguments - Utility Functions and Standard Libraries

Topics:

1. `main` with parameters and command-line arguments
 2. Input/output functions and file management
 3. Main functions of `string.h`
 4. String/number conversion functions
 5. Random number generation
-

Section 1 — The `main` Function with Parameters

Standard Form of `main`

```
int main(int argc, char *argv[])
```

Parameters:

- `argc` → number of command-line arguments
 - `argv` → array of pointers to strings containing arguments
-

Structure of `argv`

Example command:

```
./program file.txt 10
```

Content:

Element	Value
<code>argv[0]</code>	<code>"/.program"</code>
<code>argv[1]</code>	<code>"file.txt"</code>
<code>argv[2]</code>	<code>"10"</code>

Example: Printing Command-Line Arguments

```
#include <stdio.h>

int main(int argc, char *argv[]) {

    for (int i = 0; i < argc; i++) {
        printf("argv[%d] = %s\n", i, argv[i]);
    }

    return 0;
}
```

Example: Sum of Two Numbers from Command Line

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {

    if (argc != 3) {
        printf("Usage: ./program n1 n2\n");
        return 1;
    }

    int a = atoi(argv[1]);
    int b = atoi(argv[2]);

    printf("Sum = %d\n", a + b);

    return 0;
}
```

PROF

Section 2 — Input/Output and File Management

Streams in C

- `stdin` → standard input
- `stdout` → standard output
- `stderr` → standard error

Files are manipulated through pointers of type `FILE*`.

Opening and Closing Files

```
FILE *fp;  
fp = fopen("data.txt", "r");
```

Modes:

Mode	Meaning
"r"	read
"w"	write
"a"	append

Close file:

```
fclose(fp);
```

Reading from stdin with `fgets`

```
#include <stdio.h>  
  
int main() {  
  
    char name[100];  
  
    printf("Enter your name: ");  
    fgets(name, sizeof(name), stdin);  
  
    printf("Hello %s", name);  
  
    return 0;  
}
```

- `stdin` behaves like a normal input file
- `fgets` safely reads an entire line

Reading a File Line by Line with `fgets`

```
#include <stdio.h>  
  
int main() {  
  
    FILE *fp = fopen("data.txt", "r");
```

```

char line[100];

while (fgets(line, sizeof(line), fp) != NULL) {
    printf("%s", line);
}

fclose(fp);
return 0;
}

```

Semantics of `fgets`

```
fgets(buffer, size, file)
```

- Reads at most `size - 1` characters
- Stops at:
 - newline
 - end of file
 - size limit
- Adds terminating `\0`

Reading a File Character by Character with `fgetc`

```

#include <stdio.h>

int main() {
    FILE *fp = fopen("data.txt", "r");

    int c;

    while ((c = fgetc(fp)) != EOF) {
        putchar(c);
    }

    fclose(fp);
    return 0;
}

```

Reading from stdin Character by Character with `getchar`

```
#include <stdio.h>

int main() {

    int c;

    printf("Type characters (q to quit):\n");

    while ((c = getchar()) != 'q') {
        putchar(c);
    }

    return 0;
}
```

- `getchar()` reads one character from `stdin`
- Useful for simple parsers and interactive input

Reading with `fscanf`

```
FILE *fp = fopen("data.txt", "r");

int x;
float y;

fscanf(fp, "%d %f", &x, &y);
```

Summary of Input Functions

PROF

Function	Purpose
<code>scanf</code>	formatted input from <code>stdin</code>
<code>fscanf</code>	formatted input from file
<code>fgets</code>	line-oriented input
<code>fgetc</code>	single character input from file
<code>getchar</code>	single character input from <code>stdin</code>

Section 3 — Main Functions of `string.h`

The `string.h` Library

```
#include <string.h>
```

Provides functions to manipulate strings.

strlen — String Length

```
char s[] = "hello";  
printf("%lu\n", strlen(s));
```

Output:

5

strcpy — Copy String

```
char src[] = "hello";  
char dst[100];  
  
strcpy(dst, src); // copies "hello" into dst
```

strcat — Concatenate Strings

```
char s1[100] = "hello ";  
char s2[] = "world";  
  
strcat(s1, s2); // s1 contains "hello world"
```

Result:

hello world

strcmp — Compare Strings

```
if (strcmp(s1, s2) == 0)
    printf("Equal\n");
```

Return value:

Value	Meaning
0	equal
<0	first smaller
>0	first larger

strchr — Search Character

```
char *p = strchr("hello", 'e');
```

- Returns pointer to first occurrence of 'e'

strstr — Search Substring

```
char *p = strstr("hello world", "world");
```

Summary of Main `string.h` Functions

PROF

Function	Description
strlen	length
strcpy	copy
strcat	concatenate
strcmp	compare
strchr	search character
strstr	search substring

Section 4 — String/Number Conversion Functions

Converting String to Integer: `atoi`

```
char s[] = "123";
int x = atoi(s);
```

Converting String to Float: `atof`

```
char s[] = "3.14";
float x = atof(s);
```

Safer Conversions: `strtol`

```
char s[] = "123";
long x = strtol(s, NULL, 10);
```

Parameters:

- string
- end pointer
- numeric base

Parsing Data from Strings with `sscanf`

`sscanf` works like `scanf`, but reads formatted data from a string stored in memory.

Syntax:

```
sscanf(string, format, variables...);
```

Example of `sscanf`

```
#include <stdio.h>

int main() {

    char buffer[] = "25 3.14";

    int a;
    float b;
```



```
    sscanf(buffer, "%d %f", &a, &b);

    printf("a = %d\n", a);
    printf("b = %f\n", b);

    return 0;
}
```

Typical Uses of `sscanf`

- parsing lines previously read with `fgets`
- decoding structured text
- extracting multiple values from a string

Relationship Between Input Functions

Function	Reads From
<code>scanf</code>	<code>stdin</code>
<code>fscanf</code>	file
<code>sscanf</code>	string in memory

Converting Numbers to Strings: `sprintf`

```
char buffer[100];
int x = 42;

sprintf(buffer, "%d", x);
```

PROF

Example: Building Strings Dynamically

```
char message[100];
int score = 95;

sprintf(message, "Score = %d", score);
```

Summary of Conversion Functions

Function	Description
atoi	string → int
atof	string → float
strtol	robust string → integer
sscanf	formatted parsing from string
sprintf	number → string

Section 5 — Random Numbers

The `rand` Function

```
#include <stdlib.h>

int x = rand();
```

- Returns a pseudo-random integer

Limiting the Range

```
int x = rand() % 10;
```

Produces values:

0–9

Initializing the Random Generator with `srand`

```
#include <time.h>

srand(time(NULL));
```

- Different sequence each execution

Complete Example

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main() {

    srand(time(NULL));

    for (int i = 0; i < 5; i++) {
        printf("%d\n", rand() % 100);
    }

    return 0;
}
```

Random Floating-Point Number

```
float x = (float) rand() / RAND_MAX;
```

- Generates a number between 0 and 1

Summary of Random Functions

Function	Description
rand	pseudo-random integer
srand	initializes generator
time	obtains current time